

Four blocks, one direction.

Each block calls only the one below it. CI fails the build otherwise.

Routes

Pages, layouts and route handlers in app/



Feature slices

One folder per domain, one public entry



Shared lib

Database, auth and helpers. Never a feature

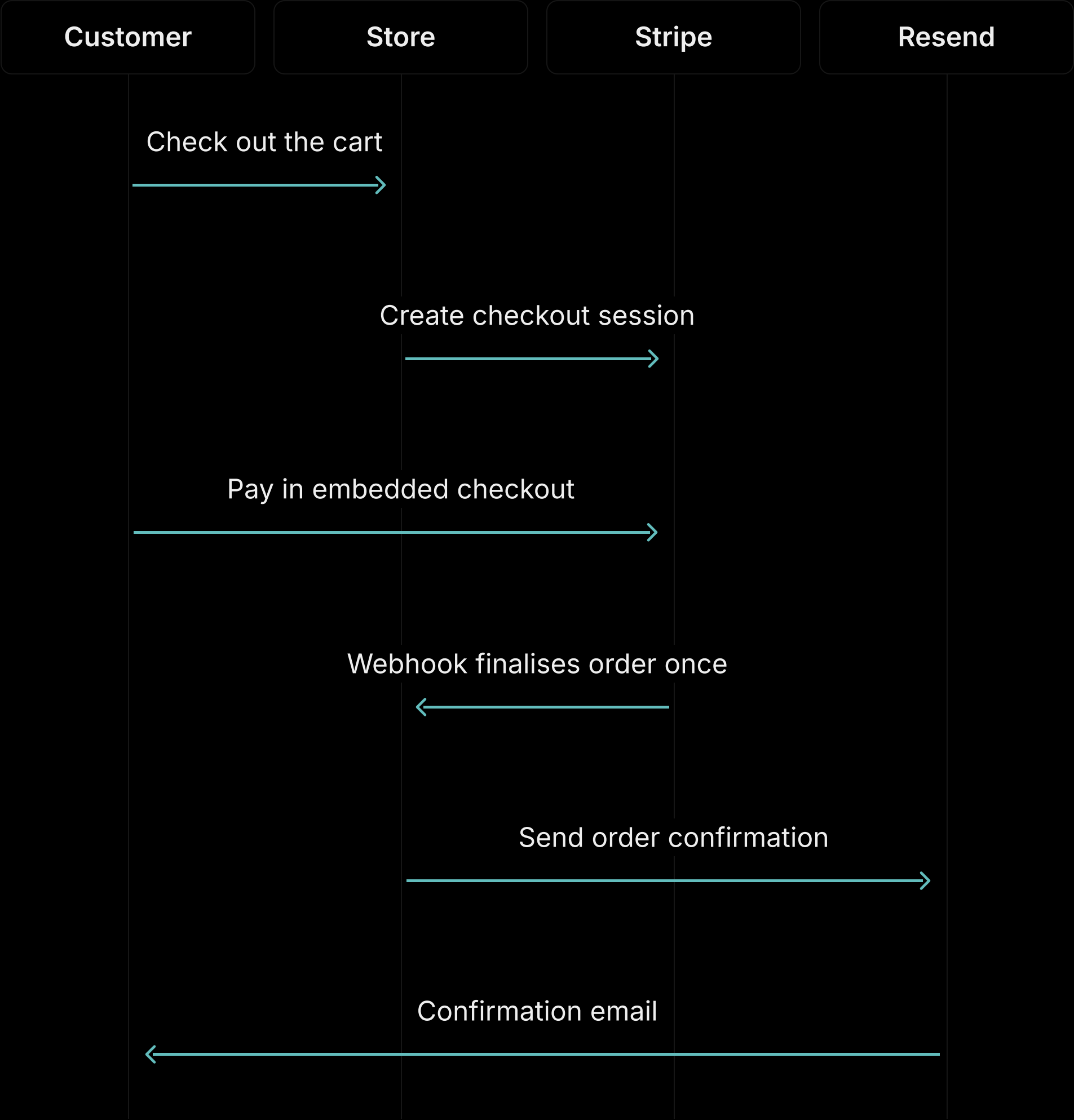


Services

Neon, Stripe and Resend behind adapters

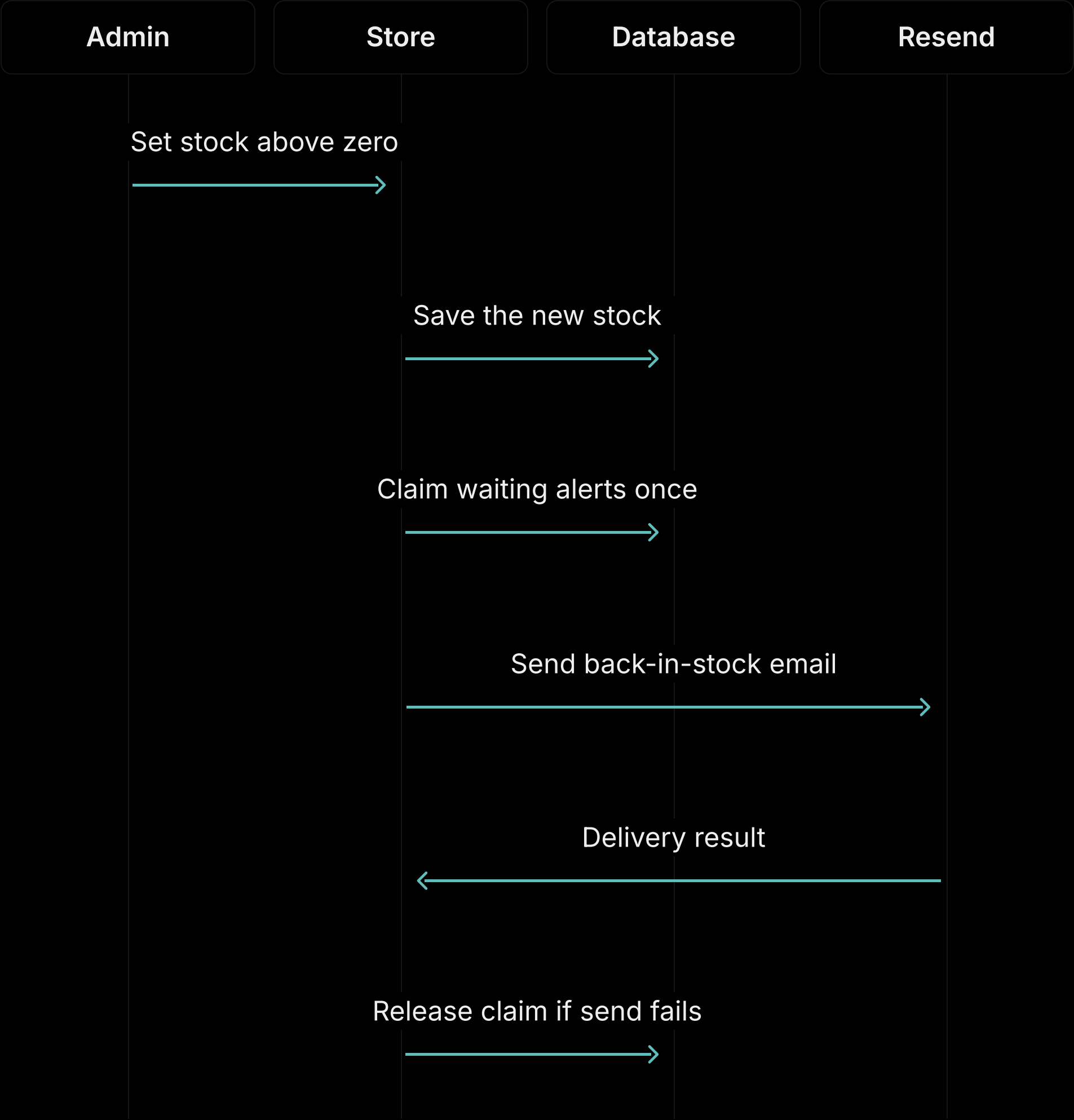
From cart to confirmation email.

The webhook, not the browser, decides an order is paid.



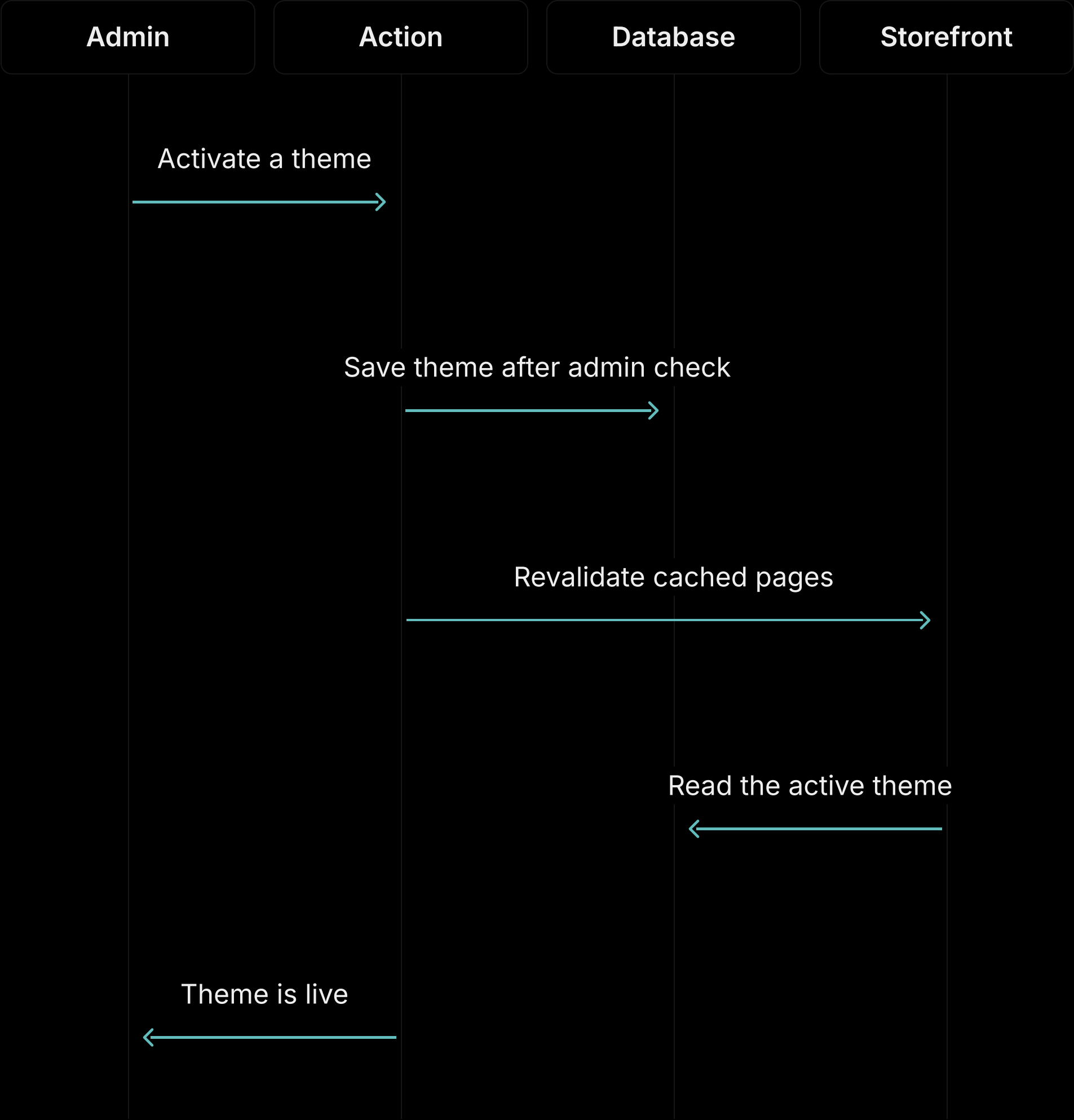
Restock to alert, claimed once.

A claim stops double sends. A failed send hands it back.



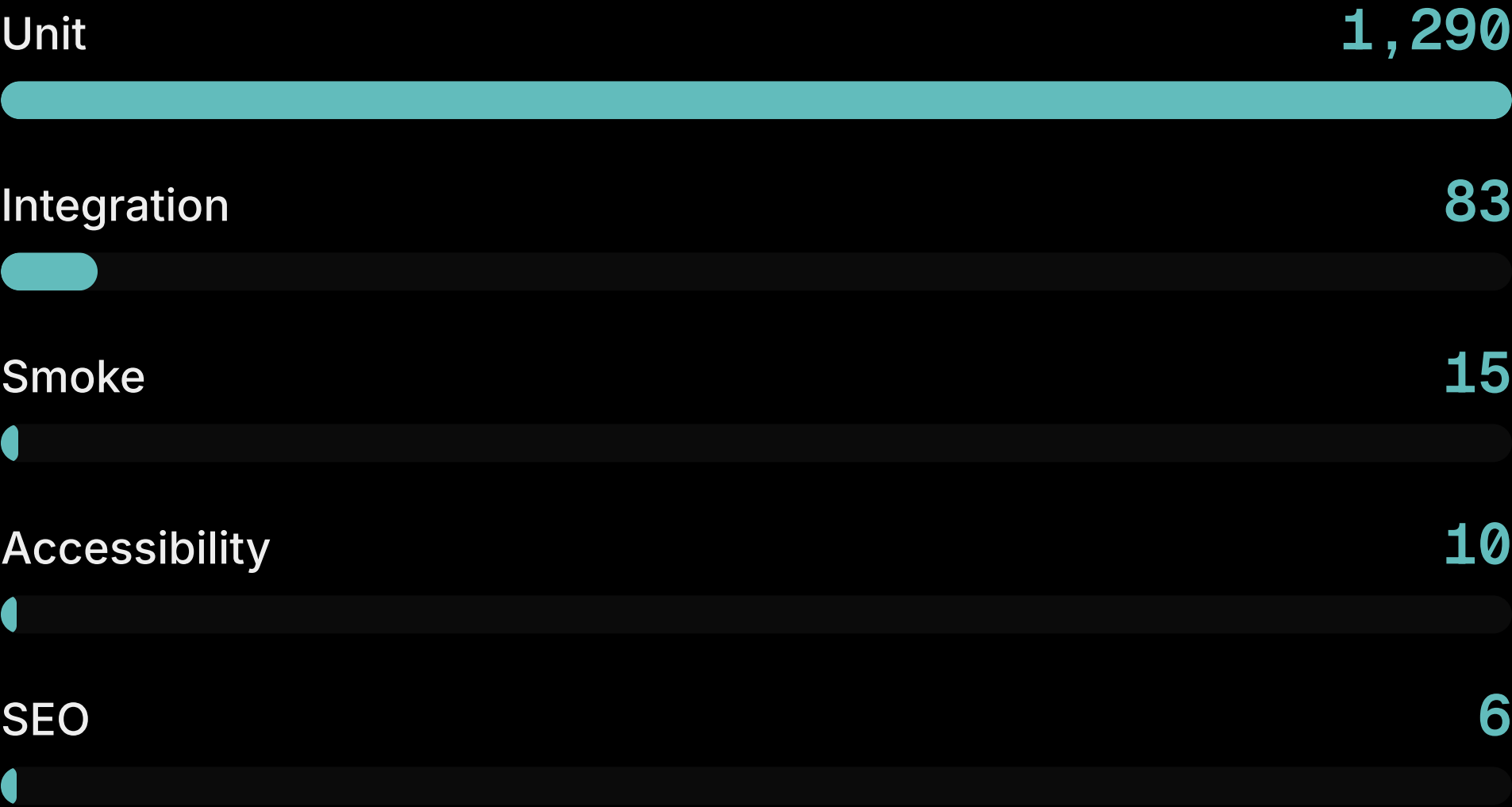
One click, a new storefront theme.

Only the server action writes, and it checks the admin first.



Many fast tests, a few slow ones.

Counted on every run, widest layer first.



Coverage can only go up.

CI fails if coverage drops below the committed baseline.

Lines covered

Baseline, then today

~~35.7~~ → **37.6**

Branches covered

Baseline, then today

~~27.4~~ → **29.0**

— LET'S TALK

Hiring for a build like this?

The case study walks through the architecture, the tests and the trade-offs behind every decision.



Scan to read

CASE STUDY

[momo-audio.adudev.co.uk](https://momo-audio.adudev.co.uk/docs/case-study-email-platform)
[/docs/case-study-email-platform](https://momo-audio.adudev.co.uk/docs/case-study-email-platform)

Open to roles and contract work. Scan the code or email me.

admin@adudev.co.uk